Atividade: Desenvolver um algoritmo

Introdução

Um algoritmo é um conjunto de etapas que pode ser usado para resolver um problema. Analistas de segurança desenvolvem algoritmos para fornecer as soluções necessárias para o seu trabalho. Por exemplo, um analista pode trabalhar com usuários que lhe trazem dispositivos. O analista pode precisar de um algoritmo que primeiro verifique se um usuário tem autorização para acessar o sistema e, em seguida, verifique se o dispositivo que ele trouxe é o que lhe foi atribuído.

Neste laboratório, você desenvolverá um algoritmo em Python que automatiza esse processo.

Dicas para concluir este laboratório

Cenário

Neste laboratório, você trabalhará como analista de segurança e será responsável por desenvolver um algoritmo que conecta usuários aos seus dispositivos atribuídos. Você escreverá um código que indica se um usuário foi aprovado no sistema e se entregou seu dispositivo atribuído à equipe de segurança.

Tarefa 1

Você trabalhará com uma lista de nomes de usuário aprovados, juntamente com uma lista dos dispositivos aprovados atribuídos a esses usuários. Os elementos das duas listas estão sincronizados. Em outras palavras, o usuário no índice `0` da lista `approved_users` utiliza o dispositivo no índice `0` da lista `approved_devices`. Posteriormente, isso permitirá que você verifique se o nome de usuário e o ID do dispositivo inseridos por um usuário correspondem.

Primeiro, para explorar como os índices em listas funcionam, execute a seguinte célula de código como está e observe a saída. Em seguida, substitua cada `0` índice por outro e execute a célula novamente para observar o que acontece.

Em [7]:
```
# Atribua `approved_users` a uma lista de nomes de usuário aprovados

usuários_aprovados = [ "elarson" , "bmoreno" , "tshah" , "sgilmore" , "eraab" ]

# Atribua `approved_devices` a uma lista de IDs de dispositivos que correspondam aos nomes de usuário em `approved_users`

dispositivos_aprovados = [ "8rp2k75" , "hl0s5o1" , "2ye3lzg" , "4n482ts" , "a307vir" ]

# Display the element at the specified index in `approved_users`

print(approved_users[3])

# Display the element at the specified index in `approved_devices`

print(approved_devices[3])
```

```
sgilmore
4n482ts
```

Question 1

What did you observe about the output when `approved_users[0]` is displayed and when `approved_devices[0]` is displayed? What happens when you replace each `0` with another index?

Para a primeira pergunta, o output exibe o primeiro item de cada lista (elarson e 8rp2k75) porque o índice 0 acessa o início da sequência; ao trocar o índice, o Python exibe os itens correspondentes às novas posições escolhidas em ambas as listas.

Task 2

There's a new employee joining the organization, and they need to be provided with a username and device ID. In the following code cell, you are given a username and device ID of this new user, stored in the variables `new_user` and `new_device`, respectively. Use the `.append()` method to add these variables to the `approved_users` and `approved_devices` respectively. Afterwards, display the `approved_users` and `approved_devices` variables to confirm the added information. Be sure to replace each `### YOUR CODE HERE ###` with your own code before you run the following cell.

In [9]:
```
# Assign `approved_users` to a list of approved usernames

approved_users = ["elarson", "bmoreno", "tshah", "sgilmore", "eraab"]

# Assign `approved_devices` to a list of device IDs that correspond to the
usernames in `approved_users`

approved_devices = ["8rp2k75", "hl0s5o1", "2ye3lzg", "4n482ts", "a307vir"]

# Assign `new_user` to the username of a new approved user

new_user = "gesparza"

# Assign `new_device` to the device ID of the new approved user

new_device = "3rcv4w6"

# Add that user's username and device ID to `approved_users` and `approved_
devices` respectively

approved_users.append("gesparza")
approved_devices.append("3rcv4w6")

# Display the contents of `approved_users`

print(approved_users)

# Diplay the contents of `approved_devices`

print(approved_devices)
```

```
['elarson', 'bmoreno', 'tshah', 'sgilmore', 'eraab', 'gesparza']
['8rp2k75', 'hl0s5o1', '2ye3lzg', '4n482ts', 'a307vir', '3rcv4w6']
```

Hint 1

Hint 2

Question 2

After the new approved user is added, what did you observe about the output when `approved_users` is displayed and when `approved_devices` is displayed?

O novo usuário e o ID do dispositivo são colocados no fim da lista.

Task 3

An employee has left the team and should no longer have access to the system. In the following code cell, you are given the username and device ID of the user to be removed, stored in the variables `removed_user` and `removed_device` respectively. Use the `.remove()` method to remove each of these elements from the corresponding list. Afterwards, display both the `approved_users` and the `approved_devices` variables to view the removed users. Run the code and observe the results. Be sure to replace each `### YOUR CODE HERE ###` with your own code before you run the following cell.

In [10]:
```
# Assign `approved_users` to a list of approved usernames

approved_users = ["elarson", "bmoreno", "tshah", "sgilmore", "eraab", "gesp
arza"]

# Assign `approved_devices` to a list of device IDs that correspond to the
usernames in `approved_users`

approved_devices = ["8rp2k75", "hl0s5o1", "2ye3lzg", "4n482ts", "a307vir",
"3rcv4w6"]

# Assign `removed_user` to the username of the employee who has left the te
am

removed_user = "tshah"

# Assign `removed_device` to the device ID of the employee who has left the
team

removed_device = "2ye3lzg"

# Remove that employee's username and device ID from `approved_users` and `
approved_devices` respectively

approved_users.remove("tshah")
approved_devices.remove("2ye3lzg")

# Display `approved_users`

print(approved_users)

# Diplay `approved_devices`

print(approved_devices)
```

```
['elarson', 'bmoreno', 'sgilmore', 'eraab', 'gesparza']
['8rp2k75', 'hl0s5o1', '4n482ts', 'a307vir', '3rcv4w6']
```

Hint 1

Hint 2

Question 3

After the user who left the team is removed, what did you observe about the output when `approved_users` is displayed and when `approved_devices` is displayed?

O nome do usuário e seu ID são removido da lista.

Task 4

As part of verifying a user's identity in the system, you'll need to check if the user is one of the approved users. Write a conditional statement that verifies if a given username is an element of the list of approved usernames. If it is, display `"The user _____ is approved to access the system."` . Otherwise, display `"The user _____ is not approved to access the system."` . Be sure to replace each `### YOUR CODE HERE ###` with your own code before you run the following cell.

In [16]:
```
# Assign `approved_users` to a list of approved usernames

approved_users = ["elarson", "bmoreno", "sgilmore", "eraab", "gesparza"]

# Assign `approved_devices` to a list of device IDs that correspond to the
usernames in `approved_users`

approved_devices = ["8rp2k75", "hl0s5o1", "4n482ts", "a307vir", "3rcv4w6"]

# Assign `username` to a username

username = "sgilmore"

# Conditional statement
# If `username` belongs to `approved_users`, then display "The user _____
is approved to access the system."
# Otherwise display "The user _____ is not approved to access the system."
if username in approved_users:
    print("The username", username, "is approved to access the system.")
else:
    print("is not approved to access the system")
```

```
The username sgilmore is approved to access the system.
```

Hint 1

Hint 2

Hint 3

Question 4

What message do you observe in the output when `username` is `"sgilmore"` ?

The username sgilmore is approved to access the system.

Task 5

The next part of the algorithm uses the `.index()` method to find the index of `username` in the `approved_users` and store that index in a variable named `ind` .

When used on a list, the `.index()` method will return the position of the given value in the list.

Add a statement to display `ind` in the following code cell to explore the value it contains. Be sure to replace the `### YOUR CODE HERE ###` with your own code before you run the following cell.

In [17]:
```
# Assign `approved_users` to a list of approved usernames

approved_users = ["elarson", "bmoreno", "sgilmore", "eraab", "gesparza"]

# Assign `approved_devices` to a list of device IDs that correspond to the
usernames in `approved_users`

approved_devices = ["8rp2k75", "hl0s5o1", "4n482ts", "a307vir", "3rcv4w6"]

# Assign `username` to a username

username = "sgilmore"

# Assign `ind` to the index of `username` in `approved_users`

ind = approved_users.index(username)

# Display the value of `ind`

print(ind)
```

```
2
```

Hint 1

Question 5

What do you observe from the output when `username` is `"sgilmore"` ?

2

Task 6

This task will allow you to build your understanding of list operations for the algorithm that you'll eventually build. It will demonstrate how you can find an index in one list and then use this index to display connected information in another list. First, use the `.index()` method again to find the index of `username` in the `approved_users` and store that in a variable named `ind`. Then, connect `ind` to the `approved_devices` and display the device ID located at the index `ind`. Afterwards, run the cell to observe the result. Be sure to replace each `### YOUR CODE HERE ###` with your own code before you run the following cell.

In [20]:
```
# Assign `approved_users` to a list of approved usernames

approved_users = ["elarson", "bmoreno", "sgilmore", "eraab", "gesparza"]

# Assign `approved_devices` to a list of device IDs that correspond to the
usernames in `approved_users`

approved_devices = ["8rp2k75", "hl0s5o1", "4n482ts", "a307vir", "3rcv4w6"]

# Assign `username` to a username

username = "sgilmore"

# Assign `ind` to the index of `username` in `approved_users`

ind = approved_users.index(username)

# Display the device ID at the index that matches the value of `ind` in `ap
proved_devices`

print(approved_devices[ind])
```

```
4n482ts
```

Hint 1

Hint 2

Question 6

What do you observe from the output when `username` is `"sgilmore"`?

4n482ts

Task 7

Your next step in creating the algorithm is to determine if a username and device ID correspond. To do this, write a conditional that checks if the `username` is an element of the `approved_devices` and if the `device_id` stored at the same index as `username` matches the `device_id` entered. You'll use the logical operator `and` to connect the two conditions. When both conditions evaluate to `True`, display a message that the username is approved and another message that the user has their assigned device. Be sure to replace each `### YOUR CODE HERE ###` with your own code before you run the following cell.

In [23]:
```
# Assign `approved_users` to a list of approved usernames

approved_users = ["elarson", "bmoreno", "sgilmore", "eraab", "gesparza"]

# Assign `approved_devices` to a list of device IDs that correspond to the
usernames in `approved_users`

approved_devices = ["8rp2k75", "hl0s5o1", "4n482ts", "a307vir", "3rcv4w6"]

# Assign `username` to a username

username = "sgilmore"

# Assign `device_id` to a device ID

device_id = "4n482ts"

# Assign `ind` to the index of `username` in `approved_users`

ind = approved_users.index(username)

# Conditional statement
# If `username` belongs to `approved_users`, and if the device ID at `ind`
in `approved_devices` matches `device_id`,
# then display a message that the username is approved,
# followed by a message that the user has the correct device

if username in approved_users and approved_devices[ind] == device_id:
    print("The username", username, "is approved to access the system.")
    print(device_id, "is the assigned device for", username)
```

```
The username sgilmore is approved to access the system.
4n482ts is the assigned device for sgilmore
```

Hint 1

Hint 2

Question 7

What do you observe from the output when `username` is `"sgilmore"` and `device_id` is `"4n482ts"` ?

The username sgilmore is approved to access the system. 4n482ts is the assigned device for sgilmore

Task 8

It would also be helpful for users to receive messages when their username is not approved or their device ID is incorrect.

Add to the code by writing an `elif` statement. This `elif` statement should run when the `username` is part of the `approved_users` but the `device_id` doesn't match the corresponding device ID in the `approved_devices` . The statement should also display two messages conveying that information.

Be sure to replace each `### YOUR CODE HERE ###` with your own code before you run the following cell.

(After you run the code once with a `device_id` of `"4n482ts"` , you might want to explore what happens if you assign a different value to `device_id` .)

In [32]:
```
# Assign `approved_users` to a list of approved usernames

approved_users = ["elarson", "bmoreno", "sgilmore", "eraab", "gesparza"]

# Assign `approved_devices` to a list of device IDs that correspond to the
usernames in `approved_users`

approved_devices = ["8rp2k75", "hl0s5o1", "4n482ts", "a307vir", "3rcv4w6"]

# Assign `username` to a username

username = "sgilmore"

# Assign `device_id` to a device ID

device_id = "4n482ts"

# Assign `ind` to the index of `username` in `approved_users`

ind = approved_users.index(username)

# If statement
# If `username` belongs to `approved_users`, and if the element at `ind` in
`approved_devices` matches `device_id`,
# then display a message that the username is approved,
# followed by a message that the user has the correct device

if username in approved_users and device_id == approved_devices[ind]:
    print("The user", username, "is approved to access the system.")
    print(device_id, "is the assigned device for", username)

# Elif statement
# Handles the case when `username` belongs to `approved_users` but element
at `ind` in `approved_devices` does not match `device_id`,
# and displays two messages accordingly

elif username in approved_users and device_id != approved_devices[ind]:
    print("The user", username, "is approved to access the system, but", de
vice_id, "is not their assigned device.")
```

```
The user sgilmore is approved to access the system.
4n482ts is the assigned device for sgilmore
```

Hint 1

Hint 2

Question 8

What do you observe from the output when `username` is `"sgilmore"` and `device_id` is `"4n482ts"` ?

The user sgilmore is approved to access the system. 4n482ts is the assigned device for sgilmore

Task 9

In this task, you'll complete your algorithm by developing a function that uses some of the code you've written in earlier tasks. This will automate the login process.

There are multiple ways to use conditionals to automate the login process. In the following code, a nested conditional is used to achieve the goals of the algorithm. There is a conditional statement inside of another conditional statement. The outer conditional handles the case when the `username` is approved and the case when `username` is not approved. The inner conditional, which is placed inside the first `if` statement, handles the case when the `username` is approved and the `device_id` is correct, as well as the case when the `username` is approved and the `device_id` is incorrect.

To complete this task, you must define a function named `login` that takes in two parameters, `username` and `device_id` . Afterwards, call the function and pass in different username and device ID combinations to experiment and observe the function's behavior. Be sure to replace the `### YOUR CODE HERE ###` with your own code before you run the following cell.

In [39]:
```
# Assign `approved_users` to a list of approved usernames

approved_users = ["elarson", "bmoreno", "sgilmore", "eraab", "gesparza"]

# Assign `approved_devices` to a list of device IDs that correspond to the
usernames in `approved_users`

approved_devices = ["8rp2k75", "hl0s5o1", "4n482ts", "a307vir", "3rcv4w6"]

# Define a function named `login` that takes in two parameters, `username`
and `device_id`

def login(username, device_id):

    # If `username` belongs to `approved_users`,

    if username in approved_users:

        # then display "The user _____ is approved to access the system.",

        print("The user", username, "is approved to access the system.")

        # assign `ind` to the index of `username` in `approved_users`,

        ind = approved_users.index(username)

        # and execute the following conditional
        # If `device_id` matches the element at the index `ind` in `approve
d_devices`,

        if device_id == approved_devices[ind]:

          # then display "_____ is the assigned device for _____"

          print(device_id, "is the assigned device for", username)

        # Otherwise,

        else:

          # display "_____ is not their assigned device"

          print(device_id, "is not their assigned device.")

    # Otherwise (part of the outer conditional and handles the case when `u
sername` does not belong to `approved_users`),

    else:

        # Display "The user _____ is not approved to access the system."

        print("The username", username, "is not approved to access the syst
em.")

# Call the function you just defined to experiment with different username
and device_id combinations

login("sgilmore", "4n482ts")
login("akjiagu","hl0s5o1" )
login("bmoreno", "a154v1")
```

```
The user sgilmore is approved to access the system.
4n482ts is the assigned device for sgilmore
The username akjiagu is not approved to access the system.
The user bmoreno is approved to access the system.
a154v1 is not their assigned device.
```

Hint 1

Hint 2

Hint 3

Question 9

After Python enters the inner conditional, what happens when the `device_id` is correct, and what happens when the `device_id` is incorrect?

Se o device_id estiver correto: O Python executa a instrução que está dentro do primeiro bloco do if interno. Ele exibe a mensagem confirmando que aquele é o dispositivo atribuído ao usuário (ex: "4n482ts is the assigned device for sgilmore").

Se o device_id estiver incorreto: O Python pula a instrução do if e vai direto para o bloco else interno. Ele exibe a mensagem de erro informando que, embora o usuário exista, aquele dispositivo específico não pertence a ele (ex: "12345 is not their assigned device.").

Conclusion

What are your key takeaways from this lab?

Automação com Funções: Aprendi que transformar um processo de login em uma função (def login) permite automatizar a verificação de segurança de forma rápida e reutilizável para múltiplos usuários.

Camadas de Segurança (Condicionais Aninhadas): Compreendi como usar um if dentro de outro para criar uma validação em duas etapas: primeiro o nome de usuário (identidade) e depois o ID do dispositivo (integridade).

Sincronização por Índices: Entendi que listas paralelas podem ser "conectadas" através de um índice comum (ind), garantindo que o sistema saiba exatamente qual dispositivo pertence a cada usuário específico.

Gestão Dinâmica: O uso de métodos como .append() e .remove() mostrou como atualizar permissões de acesso em tempo real, refletindo mudanças na equipe (entradas e saídas de funcionários).